

STOP CLICKING "ALLOW"

DON'T JUST TRUST THE AI. TRACE IT.

**15 documentation habits that turn AI-assisted coding
from a gamble into a system you actually control.**

A FIELD GUIDE FOR DEVELOPERS WORKING WITH AI

Every "Allow" click is a decision. Most people make it blind. This guide gives you the paper trail to make it informed — before, during, and after the AI touches your code.

AI coding assistants move fast. They propose a change, you hit "Allow," and the context that led there — the reasoning, the tradeoffs, the parts of the codebase it touched — evaporates the moment the session ends.

Next session, the AI starts from zero. So do you. The bugs get harder to trace. The "why" behind every decision gets lost. You're not directing the work anymore — you're just approving it.

The fix isn't slowing down. It's writing things down. Fifteen habits, five documents, zero blind trust.

5

CORE FILES TO KEEP

15

HABITS IN THIS GUIDE

0

BLIND ALLOWS

01

THE CORE FIVE FILES

These five documents form the backbone of any serious AI-assisted workflow. Keep them next to your code, not in your head.



1

HANDOVER
FILES

CONTINUITY

Write context incrementally, in a file the AI reads at the start of every session. Not a dump of everything — a living record of where things stand right now: what's done, what's in progress, what's broken, what to avoid.

WHY IT MATTERS

Every new AI session starts with amnesia. A handover file is the difference between "re-explain the entire project" and "here's exactly where we left off."

2

DECISIONS.MD

RATIONALE

Log every meaningful decision the AI makes while changing your code — and the reasoning behind it. Why this library over that one. Why this pattern. Why this tradeoff was accepted.

WHY IT MATTERS

Code shows what changed. Decisions.md shows why. Six months later, that "why" is the only thing that saves you from re-litigating settled arguments.

3

EXPLICIT COMMENTS

READABILITY

Require the AI to comment non-obvious logic as it writes it — not restating the code, but explaining the flow: what this block is for, what calls into it, what assumes it exists.

WHY IT MATTERS

Comments are cheap insurance. They let you (or the next AI session) understand intent without reverse-engineering it from scratch.

4

FLOW.MD

TRACEABILITY

Document how execution actually travels — between files, functions, and modules. What calls what, in what order, and exactly which parts of that path you're modifying right now.

WHY IT MATTERS

Bugs live in the gaps between files. If you can't see the flow, you can't see where it breaks — and neither can the AI.

5

BUG.MD / FEATURE.MD

FULL TRACE

For every bug or feature, keep one file that traces it start to finish: how it was found or scoped, what was tried, what worked, what didn't, and how it was verified.

WHY IT MATTERS

Anyone — human or AI — should be able to read this file cold and know exactly how to pick up where it left off.

02

GUARDRAIL DOCUMENTS

Beyond tracking what happened, these files control what's allowed to happen — before the AI ever touches a line.



6

ARCHITECTURE.MD

BIG PICTURE

A high-level map of the system: modules, services, how data moves between them. Not implementation detail — the shape of the thing, so the AI doesn't have to re-derive it every session.

WHY IT MATTERS

Without a map, every session starts by guessing the terrain. With one, the AI can reason about impact before it writes a single line.

7

CONSTRAINTS.MD

BOUNDARIES

A short, explicit list of what the AI must never do — don't touch the payment module, always use the repository pattern, no new dependencies without asking, and so on.

WHY IT MATTERS

"Allow" should never mean "allow anything." Constraints turn permission into scoped permission.

8

TEST CHECKLIST

VERIFICATION

A concrete list of what to run and check before any change counts as "done." Not a vibe check — actual commands, actual expected outputs.

WHY IT MATTERS

AI claiming success and code actually working are two different facts. This file is how you stop confusing them.

9

ROLLBACK.MD

SAFETY NET

A short plan for undoing a change if it breaks something — especially for large or risky edits. Which commit to revert to, which files to restore, what to re-check after.

WHY IT MATTERS

Confidence to let AI make bigger changes comes from knowing exactly how to reverse them if they go wrong.

03

REVIEW HABITS

Documents only work if you actually read them. These habits make sure a human stays in the loop.



10

READ THE
DIFF

DISCIPLINE

Never accept a change based on the AI's summary of it. Read the actual diff — line by line, every time — no matter how small the request seemed.

WHY IT MATTERS

Summaries can be wrong or incomplete. Diffs can't lie about what actually changed.

11

ASK "WHY" BEFORE
"WHAT"

PLAN FIRST

Before letting the AI implement anything, make it explain its plan and approach first. Catch bad reasoning while it's still a paragraph, not after it's 200 lines of code.

WHY IT MATTERS

It's far cheaper to correct a flawed plan than to unwind a flawed implementation.

SCOPE CONTROL

One logical change per request. Not "fix my whole auth system" — break it down. Smaller asks are easier to verify, and mistakes stay traceable to a single step.

WHY IT MATTERS

Large, vague requests produce large, vague diffs. Nobody reviews those properly — not even the AI.

04

META HABITS

The habits that hold the whole system together, session after session.



13

SESSION HANDOFF
SUMMARY

RITUAL

End every session with a five-line note: what we did, what's left, what to watch out for. Takes 30 seconds. Saves the next session from starting cold.

WHY IT MATTERS

This is the cheapest habit on the list and often the highest-leverage one.

14

VERSION-PIN YOUR
CONTEXT

TRACEABILITY

Note which AI model or version made which decision. Behavior shifts between versions — knowing which one reasoned through a given change matters when you're debugging it later.

WHY IT MATTERS

"The AI did this" isn't a full answer once you're juggling multiple models across a project's lifetime.

THE REAL RULE

If you can't explain what the code does in your own words, you're not ready to accept it — no matter how thorough the documentation is. Docs support your understanding. They don't replace it.

WHY IT MATTERS

This is the whole point. Every file above exists to get you here — not to give you an excuse to skip it.

STOP APPROVING. START DIRECTING.

01 Handover.md — where things stand

02 Decisions.md — why, not just what

03 Explicit comments — inline intent

04 Flow.md — how execution travels

05 Bug.md / Feature.md — start to finish

06 Architecture.md — the system map

07 Constraints.md — what's off-limits

08 Test checklist — proof, not claims

09 Rollback.md — the way back out

10 Read every diff, every time

11 Ask why before what

12 One change per request

13 Handoff summary, every session

14 Version-pin your context

15 Own the mental model

A blind "Allow" is not a decision. This is.